

Practical - 11

1. Race Condition

```
harshika@siri:~$ gedit race_counter.c
** (gedit:3618): WARNING **: 09:33:09.385: Could not load Peas repository: Typelib file for namespace 'Peas', version '1.0' not found
** (gedit:3618): WARNING **: 09:33:09.385: Could not load PeasGtk repository: Typelib file for namespace 'PeasGtk', version '1.0' not found
harshika@siri:~$ gcc -Wall -pthread race_counter.c -o race_counter
harshika@siri:~$ ./race_counter
=====
RACE CONDITION DEMONSTRATION
=====
Initial counter: 0
Thread 1 started.
Thread 2 started.
Thread 3 started.
Thread 4 started.
Thread 2 finished.
Thread 1 finished.
Thread 5 started.
Thread 4 finished.
Thread 3 finished.
Thread 5 finished.
=====
RESULTS
=====
Number of threads : 5
Increments/thread : 100000
Expected counter : 500000
Actual counter : 289534
Result: RACE CONDITION DETECTED!
Some increments were lost.
=====
harshika@siri:~$ gedit race_counter.c
** (gedit:4016): WARNING **: 09:34:43.047: Could not load Peas repository: Typelib file for namespace 'Peas', version '1.0' not found
```

```
1#include <stdio.h>
2#include <stdlib.h>
3#include <pthread.h>
4
5#define NUM_THREADS 5
6#define INCREMENTS 100000
7
8/* Shared variable */
9long long counter = 0;
10
11
12/* =====
13THREAD FUNCTION
14===== */
15
16void *increment_counter(void *arg)
17{
18    int thread_id = *(int *)arg;
19
20    printf("Thread %d started.\n", thread_id);
21
22    for (int i = 0; i < INCREMENTS; i++)
23    {
24        /*
25         * RACE CONDITION:
26         * Multiple threads access and modify
27         * the same variable without synchronization.
28         */
29        counter++;
30    }
31
32    printf("Thread %d finished.\n", thread_id);
33    return NULL;
34}
35
36/* =====
37MAIN
38===== */
39
40int main()
41{
42    pthread_t threads[NUM_THREADS];
43    int thread_ids[NUM_THREADS];
44
45    printf("===== \n");
46    printf("RACE CONDITION DEMONSTRATION \n");
47    printf("===== \n");
48    printf("Initial counter: %lld\n", counter);
49}
```

2. Mutex Synchronization

```
harshika@siri:~$ gedit mutex_counter.c
** (gedit:4599): WARNING **: 09:45:24.451: Could not load Peas repository: Typelib file for namespace 'Peas', version '1.0' not found
** (gedit:4599): WARNING **: 09:45:24.451: Could not load PeasGtk repository: Typelib file for namespace 'PeasGtk', version '1.0' not found
harshika@siri:~$ gcc -Wall -pthread mutex_counter.c -o mutex_counter
harshika@siri:~$ ./mutex_counter
=====
MUTEX SYNCHRONIZATION DEMO
=====
Initial counter: 0
Thread 1 started.
Thread 2 started.
Thread 4 started.
Thread 3 started.
Thread 5 started.
Thread 1 finished.
Thread 2 finished.
Thread 4 finished.
Thread 5 finished.
Thread 3 finished.
=====
RESULTS
=====
Number of threads : 5
Increments/thread : 100000
Expected counter : 500000
Actual counter : 500000
Result: Synchronization successful!
No increments were lost.
=====
harshika@siri:~$ gedit mutex_counter.c
** (gedit:4796): WARNING **: 09:45:09.389: Could not load Peas repository: Typelib file for namespace 'Peas', version '1.0' not found
```

```
1#include <stdio.h>
2#include <stdlib.h>
3#include <pthread.h>
4
5#define NUM_THREADS 5
6#define INCREMENTS 100000
7
8/* Shared variable */
9long long counter = 0;
10
11/*
12 * Mutex used to protect the shared counter.
13 */
14pthread_mutex_t counter_mutex;
15
16
17/* =====
18THREAD FUNCTION
19===== */
20
21void *increment_counter(void *arg)
22{
23    int thread_id = *(int *)arg;
24
25    printf("Thread %d started.\n", thread_id);
26
27    for (int i = 0; i < INCREMENTS; i++)
28    {
29        /*
30         * Lock the critical section.
31         */
32        pthread_mutex_lock(&counter_mutex);
33
34        /*
35         * Critical section:
36         * Only one thread can execute this
37         * part at a time.
38         */
39        counter++;
40
41        /*
42         * Unlock after modifying counter.
43         */
44        pthread_mutex_unlock(&counter_mutex);
45    }
46
47    printf("Thread %d finished.\n", thread_id);
48    return NULL;
49}
50
51
52/* =====
```